



on Information and Systems

DOI:10.1587/transinf.2024EDP7194

Publicized:2024/10/30

This article has been accepted and published on J-STAGE in advance of copyediting. Content is final as presented.



A PUBLICATION OF THE INFORMATION AND SYSTEMS SOCIETY

The Institute of Electronics, Information and Communication Engineers

Kikai-Shinko-Kaikan Bldg., 5-8, Shibakoen 3 chome, Minato-ku, TOKYO, 105-0011 JAPAN

PAPER

Leveraging Different Boolean Function Decompositions to Reduce T-Count in LUT-based Quantum Circuit Synthesis

David CLARINO^{†a)}, Naoya ASADA^{†b)}, Atsushi MATSUO^{††c)}, *Nonmembers*,
and Shigeru YAMASHITA^{†d)}, *Senior Member*

SUMMARY

Lookup Table (LUT) based synthesis methods have recently been proposed as a way to synthesize quantum Boolean circuits in a qubit-constrained environment. Other recent research papers have demonstrated the possibility of using *relative phase* quantum circuits when compute/uncompute logic is used in tandem, reducing T-count in quantum Boolean circuits in the fault-tolerant quantum computing paradigm. Because LUT-based synthesis methods use compute/uncompute pairs on ancilla qubits, this suggests that implementing the arbitrary Boolean logic that make up the individual Boolean logic network nodes in a relative phase manner could reduce the T-count. To generate such arbitrary Boolean functions, we utilize Shannon's decomposition, Davio expansions, as well as alternating balanced and unbalanced relative phase circuits. Experimental results demonstrate that our method can reduce the T-count to an average of 24% of the existing method.

key words: *relative-phase Toffoli gates (RTOF), circuit optimization, LUT, quantum circuit synthesis*

1. Introduction

Quantum Boolean circuits [1] implement Boolean operations as quantum circuits and are common components of many quantum algorithms. To realize such quantum Boolean circuits, a Toffoli gate, which implements the Boolean AND, is an essential logic primitive in a universal gate set. These Toffoli gates have to be, in turn, composed of physically realizable gates, such as NOT, CNOT, and T-gates (e.g., the Clifford + T [2] basis gate set), as depicted in Fig. 1(a). However, in the fault-tolerant paradigm, T-gates incur much higher cost than the NOT and CNOT gates. It is therefore useful to consider ways to reduce this *T-count*, which is the number of both the T-gates and their inverse, the $T^\dagger$ -gates in the circuit.

One of the methods that has been proposed to reduce the T-count is to implement Boolean functions only up to a *relative phase* [2], which is an input dependent phase on the output. Some recent work [3–5] has proposed relative phase versions of common Boolean functions, which achieve T-count savings by removing the T-gates that only affect the

phase. However, because quantum Boolean circuits are often used as subroutines inside larger quantum algorithms, the introduced relative phase is, in general, problematic. It can only be used inside applications where the relative phase on the output is erased before the output is used elsewhere.

Both [3] and [5] have also proposed applications of relative phase circuits, such as the ones proposed in [4], as components in the composition of larger circuits. These tend to use matching pairs of relative phase gates as a possible application of relative phase circuits, because any relative phase introduced can be canceled out by simply using the inverse of the gate in instances where the effects of the gate have to be reversed. Because of these findings, logic which uses pairs of identical quantum Boolean circuits such as that produced by Lookup Table (LUT) based synthesis methods [6], thus present an interesting candidate to integrate with relative phase methods.

Recent attempts to synthesize quantum circuits using LUT-based methods have shown some promising results [6]. As intermediate nodes pass their outputs through each other into output nodes, logic produced by such a method is exactly the type that can be improved by synthesizing relative phase versions of its component logic. In addition, the fact that relative phase versions can be implemented without any additional ancilla means that it works well to synthesize Boolean functions in the qubit constrained environment that such methods are intended for.

Our research seeks to leverage recent results in relative phase implementations of Boolean functions and LUT-based quantum circuit synthesis to reduce the T-count in a quantum circuit. In particular, we propose a method to generate arbitrary Boolean functions up to a relative phase which we use to generate gates. This method uses Shannon's decomposition and Davio expansions which have previously been used to synthesize purely reversible gates [7]. We utilize them in this work to enable explicitly relative phase constructions.

Our Contribution. We propose a method that reduces the number of T-gates in a LUT-based synthesized quantum Boolean circuit by generating relative phase quantum Boolean circuits. Our contributions contain:

- A method to generate arbitrary Boolean functions up to a relative phase using Shannon's decomposition and Davio expansions, as well as balanced and unbalanced constructions of relative phase gates
- A method to generate a quantum Boolean circuit from

[†]The author is with the Graduate School of Information Science and Engineering, Ritsumeikan University, Shiga, 525-8577 Japan.

^{††}The author is with IBM Quantum, IBM Research - Tokyo, Tokyo, 103-8510 Japan.

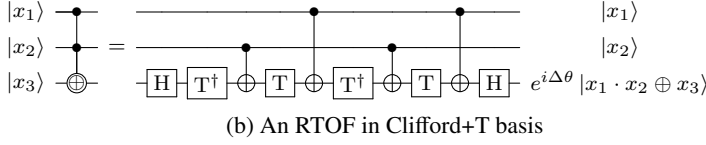
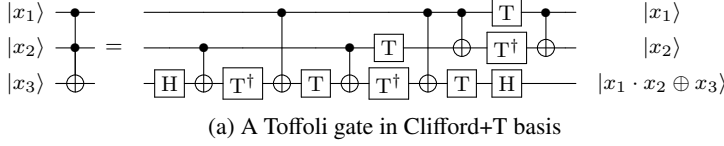
A preliminary version of this paper is presented in ICCAD 2023.

a) E-mail: dizzy@ngc.is.ritsumeikan.ac.jp

b) E-mail: accel@ngc.is.ritsumeikan.ac.jp

c) E-mail: matsuo@jp.ibm.com

d) E-mail: ger@cs.ritsumeikan.ac.jp



x_1, x_2, x_3	$\Delta\theta$
000	0
001	0
010	0
011	0
100	0
101	0
110	$+\pi/2$
111	$+\pi/2$

(c) Phases $\Delta\theta$ added by the RTOF gate in Fig. 1(b)

Fig. 1: A Toffoli gate and an RTOF gate

a Boolean logic network representation leveraging the above method to reduce the T-count

The paper is structured as follows. Section 2 gives preliminary information to understand this paper. Section 3 follows with our proposed method, including both the method for implementing functions up to a relative phase without any additional ancilla, and the method for integrating these relative phase circuits into a total function. Then, our experimental results in Section 4 compare the proposed method to a simplistic relative phase method, as well as the case which does not utilize relative phases, and find a clear advantage in its use. Finally Section 5 contains our conclusions and future work.

2. Preliminary

2.1 Quantum Boolean Circuits

In this paper, we consider *quantum Boolean circuits* [1] that implement Boolean functions. The input qubits of a quantum Boolean circuit are assigned to states $|x_n\rangle, \dots, |x_1\rangle$ when we have n input qubits. These $\{x_n, \dots, x_1\}$ are Boolean variables for the given Boolean function $F(x_n, \dots, x_1)$ that the quantum Boolean circuit implements.

An example of such a quantum Boolean operation is the Toffoli gate depicted in Fig. 1(a). Because it implements the mapping $|x_1\rangle|x_2\rangle|x_3\rangle \rightarrow |x_1\rangle|x_2\rangle|x_1 \cdot x_2 \oplus x_3\rangle$, it gives us a way to implement the Boolean AND operation, a fundamental building block for building arbitrary Boolean functions.

We implement quantum Boolean circuits using the X ($|x\rangle \mapsto |\bar{x}\rangle$), CNOT ($|x\rangle|y\rangle \mapsto |x\rangle|x \oplus y\rangle$), S ($|x\rangle \mapsto e^{i\frac{\pi}{2}}|x\rangle$), H ($|x\rangle \mapsto |0\rangle + (-1)^x|1\rangle$), and T ($|x\rangle \mapsto e^{i\frac{\pi}{4}}|x\rangle$) gates. This set is the basic set of gates in fault-tolerant quantum computing. The X, CNOT, S, and H gates are what are known as Clifford gates, which are efficiently implementable in the fault-tolerant paradigm. The T-gate, however, is not a Clifford gate, and costs substantially more to implement.

2.2 Relative-Phase Boolean Functions

A typical decomposition of the Toffoli gate into the Clifford + T basis consists of seven T-gates, as shown in Fig. 1(a). A Relative-Phase Toffoli gate (RTOF) [4], in contrast, implements the same function using only four T-gates as shown

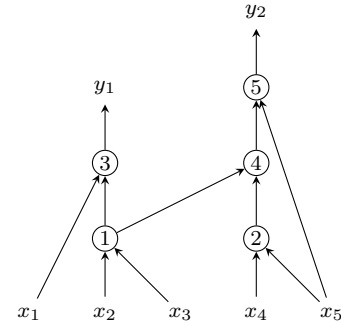


Fig. 2: A Boolean logic network

in Fig. 1(b). However, it does this at the cost of a relative phase. Note the double circle on the target of Fig. 1(b): We will be using this notation to denote a relative phase version of a quantum logic gate in this paper.

As depicted in Fig. 1(b), an RTOF gate has three inputs; the input quantum basis states are $|000\rangle$ to $|111\rangle$. Similarly to Toffoli gates, an RTOF gate swaps $|110\rangle$ and $|111\rangle$. In addition to this logic operation, RTOF gates add phases to some output states that the equivalent Toffoli gate does not. The added phases are as shown in Fig. 1(c). For example, when the input state is $|110\rangle$, a phase $e^{i\frac{\pi}{2}}$ is added.

2.3 Boolean Logic Network

A *Boolean logic network* is a simple Directed Acyclic Graph (DAG). It has nodes, edges, primary outputs, and primary inputs. Each Boolean logic network represents a Boolean function, and a *node* represents a subfunction used to decompose that Boolean function. It has zero or more *fanins*, or incoming edges, as well as zero or more *fanouts* or outgoing edges. The *primary inputs (PIs)* are nodes without fanins in the current network. The *primary outputs* are a subset of nodes in the network without fanouts. The maximum number of fanins that any node in the network has is called the network's *LUT-size*.

A Boolean function can be decomposed into such a Boolean logic network. Its nodes can subsequently be implemented as LUTs or logic gates. We see an example in Fig. 2. Here, the nodes are numbered 1-5 to abstract away the Boolean functions inside of them. The primary outputs are labeled y_1 and y_2 , and the primary inputs are labeled $x_1, \dots, x_5$. We read this graph as follows: node 1 is dependent on x_2 and x_3 , and node 2 is dependent on x_4 and x_5 .

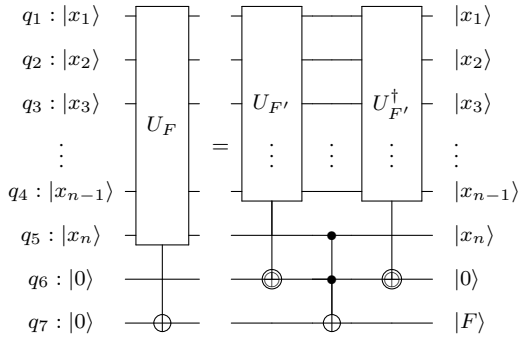


Fig. 3: Toffoli gate decomposition from [3]

node 3 is in turn dependent on node 1 and x_1 , and node 4 is dependent on both nodes 1 and 2. Finally, node 5 depends on both node 4 and x_5 .

2.4 Using RTOF Gates to Decompose MCT Gates

Existing research [3] [4] finds an advantage to using RTOF gates in synthesizing and decomposing logic functions, particularly *Multiple Control Toffoli* (MCT) Gates, which are analogous to Toffoli gates in that they apply a NOT to the target bit if the AND of all the control bits is true.

In Fig. 3, we depict a decomposition of an MCT gate from [3]. The gate U_F is a single-target quantum gate, which conducts an X operation on its target bit when the Boolean expression F is true. For an MCT gate, $F(x_n, \dots, x_1) = x_n \cdot x_{n-1} \cdots x_1$. Likewise, the component $U_{F'}$ gate acts on the ancilla qubit q_6 , and is used to decompose U_F . $U_{F'}$ implements $F'(x_{n-1}, \dots, x_1) = x_{n-1} \cdots x_1$. Likewise, $U_{F'}^\dagger$ is another single-target gate, which also implements F' . $U_{F'}^\dagger$ reverses the effects of $U_{F'}$ on q_6 , to preserve the effects of interference in the output bits. We call the action of $U_{F'}$ *compute*, and the action of $U_{F'}^\dagger$ *uncompute*. Fig. 3 implements $x_n \cdot F'$, which gives us a circuit that implements the MCT.

In [3], it was found that when decomposing larger Boolean functions, such as MCT, into compute/uncompute pairs of smaller gates that act on ancilla, it is always more advantageous to implement the smaller gates up to a relative phase. We can implement each compute/uncompute half with relative phases defined such that the relative phases cancel each other out. In Fig. 3, $U_{F'}$ turns q_6 into $e^{i\theta(x_{n-1}, \dots, x_1)} |F'\rangle$, where $\theta(x_{n-1}, \dots, x_1) \in [-\pi, \pi]$ is some phase that is dependent on the Boolean primary inputs $x_n, \dots, x_1$. F' implemented only up to a relative phase $\theta(x_{n-1}, \dots, x_1) \neq 0$ will have lower T-count than when $\theta(x_{n-1}, \dots, x_1) = 0$. Similarly, when $U_{F'}^\dagger$ implements F' up to a relative phase of $-\theta(x_n, \dots, x_1) \neq 0$. This similarly has a lower T-count than the analogous $-\theta(x_n, \dots, x_1) = 0$ case. Using the relative phase $U_{F'}$ and $U_{F'}^\dagger$ in tandem thus results in lower T-count than the same tandem without a relative phase.

To summarize:

- Pairs of computing and uncomputing gates can tolerate

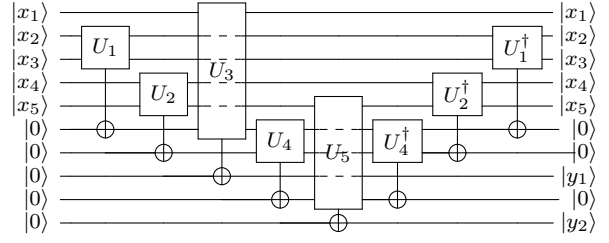


Fig. 4: The quantum circuit implementing the Boolean logic network in Fig. 2 using the LHRS

a relative phase in the compute portion if the inverse is implemented to have the opposite relative phase. This removes the need for extra T-gates that only return the phase to 0.

- Using the above, it is possible to use clean ancilla to decompose an MCT with fewer T-gates than the known methods.

2.5 LUT-based Hierarchical Reversible Logic Synthesis (LHRS)

Previous research [6] has detailed a method to use Boolean logic networks to generate quantum circuits in a manner similar to classical LUT-based logic synthesis. In this paradigm, each of the primary inputs is mapped to an input qubit, and each of the primary outputs is mapped to an output qubit. The nodes are interpreted as a lookup table, where the inputs are interpreted as indices, and the outputs are entries, according to its truth table. As a quantum circuit, each node, which represents a Boolean function F , is implemented as a single-target gate U_F , such that U_F applies the X operation to the target bit when the Boolean function F is true.

The result of the method is shown in Fig. 4. To get from Fig. 2 to Fig. 4, first, we take as input a Boolean logic network synthesized from a Boolean expression using existing Boolean logic network decomposition methods. Then this Boolean logic network is parsed by the algorithm to create a quantum circuit of single-target gates. Any intermediate node is decomposed as two single-target gates: one to compute, and the other to uncompute, acting on the same dirty ancilla. Any node that acts only on output qubits is given only one single-target gate.

Because the single-target gates act on ancilla, there is a need to uncompute them. Therefore, the flow next produces uncompute logic. This produces the compute/uncompute pair that a relative-phase based construction of the function can take advantage of.

Example 1: We start at node 1. Since this is an intermediate node, we place U_1 to act on an ancilla, where U_1 applies the X operation to the ancilla when the Boolean expression in node 1 is true. We then place its inverse $U_1^\dagger$ on the same ancilla to uncompute. Node 2 is similarly independent, so we do the same for node 2. Node 4 is dependent on both nodes 1 and 2, so we connect the target qubits from U_1 and U_2 into U_4 as input, and place U_4 , as well as its uncompute

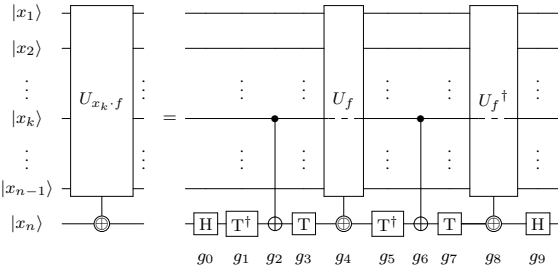
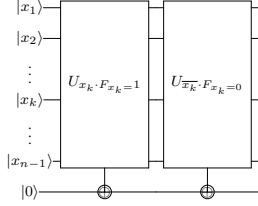
Fig. 5: A gate implementing $x_k \cdot f$ up to a relative phase

Fig. 6: Conducting the Boolean OR for Shannon's decomposition using single-target gates

gate $U_4^\dagger$, on another ancilla. Finally, nodes 3 and 5 are output nodes, so we only generate their compute logic.

Note in this diagram we denote qubits that a gate does not act on as dashed lines running across the gate, as opposed to qubits it acts on which terminate at the edges.

3. Proposed Method

3.1 Realizing Arbitrary Boolean Functions Up To A Relative Phase

To get to a point where we can realize arbitrary Boolean functions up to a relative phase, from [5], we take the circuit depicted in Fig. 5. This circuit implements the Boolean function $x_k \cdot f$, where x_k is an arbitrary variable and $f(x_n, \dots, x_{k+1}, x_{k-1}, \dots, x_1)$ is an arbitrary Boolean function. Observe that when $f = x_1$ and $x_k = x_2$, U_f and $U_f^\dagger$ can be implemented by CNOTs controlled by x_1 and x_2 respectively. We then get Fig. 1(b). This means that Fig. 5 is a generalized form of the RTOF gate that we saw earlier.

3.2 Using Shannon's Decomposition To Realize Arbitrarily Controlled Single Target Gates

Recall Shannon's decomposition $F(x_n, \dots, x_k, \dots, x_1) = x_k \cdot F_{x_k=1} + \overline{x_k} \cdot F_{x_k=0}$, where $F_{x_k=1} = F(x_n, \dots, x_k = 1, \dots, x_1)$ and $F_{x_k=0} = F(x_n, \dots, x_k = 0, \dots, x_1)$. Using the constructions mentioned in Sec. 3.1, we thus have a way to recursively construct arbitrary Boolean functions up to a relative phase.

As a prelude to our discussion, we observe that $x \cdot a + \overline{x} \cdot b = x \cdot a \oplus \overline{x} \cdot b$. Here, we have two Boolean values a and b . If we use them to create two Boolean functions $x \cdot a$ and $\overline{x} \cdot b$, related by multiplication to a Boolean value x and its inverse $\overline{x}$, respectively, we can implement the OR of $x \cdot a$ and $\overline{x} \cdot b$ as an XOR. This means that we can implement

functions analogous to $x \cdot a + \overline{x} \cdot b$ as a series of single-target gates. Suppose we have two single-target gates $U_{x_k \cdot F_{x_k=1}}$ and $U_{\overline{x_k} \cdot F_{x_k=0}}$, which implement the Boolean functions $x_k \cdot F_{x_k=1}$ and $\overline{x_k} \cdot F_{x_k=0}$ respectively. This means that to implement the function $x_k \cdot F_{x_k=1} + \overline{x_k} \cdot F_{x_k=0}$, we can place the aforementioned gates in series to compute their OR. We put them together using the construction shown in Fig. 6.

Then, from the Shannon's decomposition formula $F(x_n, \dots, x_k, \dots, x_1) = x_k \cdot F_{x_k=1} + \overline{x_k} \cdot F_{x_k=0}$, for $x_k \in \{x_n, \dots, x_1\}$. As always with Shannon's decomposition, we can decompose using an arbitrary variable order. We create a circuit by taking Fig. 5 and substituting $F_{x_k=1}$ for f , and repeat it to create a separate circuit for $F_{x_k=0}$. We then take the two circuits we just created and place them in series just as in Fig. 6. We then get the construction as shown in Fig. B3 from Appendix B. It is easy to see that this expression implements the Sum of Products (SOP) expression of Shannon's decomposition.

Note that with this decomposition, when $F_{x_k=0}$ or $F_{x_k=1}$ evaluate as "True", $\overline{x_k} \cdot F_{x_k=0}$ becomes $\overline{x_k}$ and $x_k \cdot F_{x_k=1}$ becomes x_k , respectively. Similarly, if $F_{x_k=0}$ or $F_{x_k=1}$ is "False", the products $\overline{x_k} \cdot F_{x_k=0}$ and $x_k \cdot F_{x_k=1}$ evaluate to "False". This means that these products can be implemented with at most a CNOT gate in these cases. This allows us to avoid expending any additional T-gates to implement the function here.

Example 2: We attempt to implement the function $F(x_3, x_2, x_1) = x_3 x_2 x_1 + \overline{x_3}(x_1 + x_2)$ as its relative phase equivalent.

For the $F(x_3 = 1, x_2, x_1)$ case, we note that this construction just looks like a Toffoli gate. The first stage of this decomposition looks exactly like the decomposition in Fig. 1(b). We can see that if we continue Shannon's decomposition, one or the other side of the equation will always evaluate to 0. Therefore, it is easy to see that it simplifies to the decomposition in Fig. 5, which is the decomposition in [5]. (As an aside, it generalizes it slightly as the algorithm also automatically handles the negated input case without any additional information).

For the $F(x_3 = 0, x_2, x_1)$ case, we now have something that is not a straightforward sum of products term. We now move on to the next level (note: the variable order here, $x_3 \rightarrow x_2 \rightarrow x_1$, is chosen to best illustrate the working of the method; we will treat the optimal variable order later). We see that $F(x_3 = 0, x_2 = 0, x_1) = x_1$. Following the algorithm here results in something that looks like an x_2 and x_1 controlled Toffoli gate. We then move on to $F(x_3 = 0, x_2 = 1, x_1)$ and find that it is *True*. This is one of the special cases in our algorithm, and so we can implement $x_2 \cdot F(x_3 = 0, x_2 = 1, x_1)$ as an x_2 controlled NOT gate. We then use the construction depicted in Fig. 6 to compose the two functions.

Finally, we compose $F(x_3 = 1, x_2, x_1)$ and $F(x_3 = 0, x_2, x_1)$ as in Fig. 6 again. The result is depicted in Fig. A1 in Appendix A.

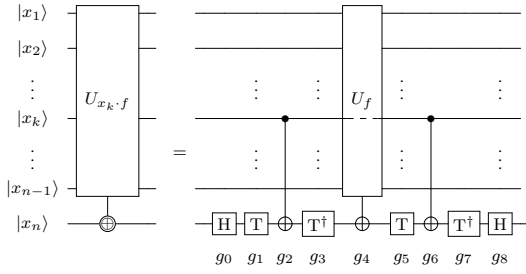


Fig. 7: An unbalanced construction implementing $x_k \cdot f$ up to a relative phase

However, as we shall see later in Section 4, some expressions are not amenable to a purely Shannon-based decomposition. Several of them have highly complex representations through Shannon’s decomposition. We must then introduce a different representation to take these into account.

3.3 Integrating Davio Expansions

As can be seen in the previous section, a significant portion of the cost comes from Boolean multiplication. (For the remainder of this paper, we will refer to *Boolean multiplication* simply as multiplication). Therefore, it is useful to consider a method that takes advantage of *Exclusive Sum of Products* (ESOP) expressions to reduce this multiplication cost. Recall the positive Davio Expansion $F = F_{x_k=0} \oplus x_k \cdot F_2$ and the corresponding negative Davio expansion $F = F_{x_k=1} \oplus \bar{x}_k \cdot F_2$ where F is some function of variables $(x_n, \dots, x_k, \dots, x_0)$, and $F_2 = F_{x_k=0} \oplus F_{x_k=1}$. These expansions have the advantage that, at every level, only one multiplication takes place, which already reduces the maximum possible T-gate use at any node from 8 to 4.

3.4 Using Balanced and Unbalanced Relative Phase Functions

To realize a savings in T-count, we again observe that Fig. 5 doubles the T-count at every level of the Shannon’s decomposition. This is because of gate g_8 in Fig. 5 which serves to uncompute the effects of gate g_4 on the target bit. Observe that the CNOT gate g_6 does the same thing with g_2 . In general, we will call these types of constructions, where all effects of CNOT and single-target gates on the target bits are reversed by uncomputing it with a similar gate, *balanced*.

We again take a different type of construction from [5], this time depicted in Fig. 7. In this construction, the analogous g_8 gate from Fig. 5 is missing such that the computation from g_4 is not reversed. Here, the target qubit is rotated by a Hadamard, acted on, and then rotated back with a Hadamard without returning the target qubit to the same computational basis states it had just after the first Hadamard. We call this type of construction, an *unbalanced* construction. In this type of construction, there is no need to double the T-count to return the target bit to its original state. However, it cannot be used recursively inside itself. That is, Fig. 7 cannot also be used to implement U_f inside itself up to a relative

phase. This is because the unbalanced constructions create a target qubit dependent relative phase. When the relative phase is dependent on the target qubit, the final Hadamard transformation, g_8 in Fig. 7, changes the function in state space as well. In the interest of keeping this paper focused on the proposed method, we refer the reader to [5] for further details. We summarize the limitations of Fig. 7’s use in our application with the following statement: unbalanced constructions can be used to implement arbitrary single-target gates (e.g. Fig. 5) used in multiplication only when done in a balanced construction.

In Appendix B, we enumerate balanced and unbalanced implementations of Shannon’s Decomposition, Positive Davio Expansion, and Negative Davio Expansion. We will be using these later to define our algorithm.

Amy et al. [5] propose a method where balanced and unbalanced constructions are alternated at different levels of the construction of an MCT to reduce the amount of times a function has to be doubled to cancel itself out. Unbalanced constructions can be used inside balanced constructions, and vice versa. We can easily extend this to our method of constructing arbitrary Boolean functions above. We start by implementing the decomposition about our first variable with an unbalanced construction, and its immediate children that are constructed recursively will have to be constructed in a balanced manner. The children of those in turn can be unbalanced, and so on.

In doing so, we can reduce the dependency from 2^{n-1} to $2^{\frac{n-1}{2}}$. While still exponential, this has a significant reduction in the regime we are working with. Merely introducing this methodology results in significant savings for larger LUT-sizes, though its effect could be diminished for smaller ones.

This has additional ramifications when combined with the Davio expansions. Recall that we have only specified a limitation for the unbalanced construction’s use within multiplication i.e. when it is used between a pair of Hadamard gates. There is no such restriction when we are not doing any multiplication. Since there is only one multiplication performed, the non-multiplied term ($F_{x_k=0}$ in the positive Davio case and $F_{x_k=1}$ in the negative Davio case) can be implemented as an unbalanced relative phase circuit, provided the parent construction can also be unbalanced. This means there is no need to double the logic inside to return the target states, and so the exponential is further mitigated.

3.5 Relative Phase Synthesis Algorithm

We now integrate Sec 3.2, Sec 3.3, and Sec 3.4 into a synthesis function up to a relative phase, defined in Algorithm 1.

First, we determine the variable order to decompose the function. As the details of the variable order computation are not important to the algorithm itself, we will defer discussion of how we determine this variable order to our discussion of the implementation in Sec. 4.

If the function is neither “True” nor “False”, the function is expanded using the Shannon, Positive Davio, or Negative Davio expansions, independently about the first variable in

Algorithm 1 relative_phase(BF: a Boolean function, balanced: Boolean indicating whether this function is balanced)

```

QC ← an empty quantum circuit with registers
Output ← the output bit of the Boolean Function
order ← the indices of a given variable order for BF.
BFvariable←value ← BF with the value of variable set to value
if BF == True then
  return NOT
else if BF == False then
  return QC ← identity map from input to output
else if BF only consists of the first variable in order then
  return CNOT
else
  k ← first variable index in order
  //The Shannon  $F_{x_k=0}$  and  $F_{x_k=1}$  alternate balanced and unbalanced
   $F_{x_k=0} \leftarrow \text{relative\_phase}(BF_{x_k \leftarrow 0}, \text{!balanced})$ 
   $F_{x_k=1} \leftarrow \text{relative\_phase}(BF_{x_k \leftarrow 1}, \text{!balanced})$ 

  //The Davio  $F_{x_k=0}$  and  $F_{x_k=1}$  inherit the parent's balanced attribute
   $(F_{x_k=0})_{dav} \leftarrow \text{relative\_phase}(BF_{x_k \leftarrow 0}, \text{balanced})$ 
   $(F_{x_k=1})_{dav} \leftarrow \text{relative\_phase}(BF_{x_k \leftarrow 1}, \text{balanced})$ 

   $F_2 \leftarrow \text{relative\_phase}(BF_{x_k=\text{first in order} \leftarrow 1} \oplus BF_{x_k=\text{first in order} \leftarrow 0}, \text{!balanced})$ 

  shannon ← compose  $F_{x_k=0}$  and  $F_{x_k=1}$  according to Fig. B3 if
  balanced is true or Fig. B4 if not.
  daviopos ← compose  $(F_{x_k=0})_{dav}$  and  $F_2$  according to Fig. B5 if
  balanced is true or Fig. B6 if not.
  davioneg ← compose  $(F_{x_k=1})_{dav}$  and  $F_2$  according to Fig. B7 if
  balanced is true or Fig. B8 if not.
  return one of [shannon, daviopos, davioneg] with the min T-count
end if
Repeat for every variable order and pick the minimum

```

the variable order. We go over constructions for them in Appendix B. The constituent subfunctions of each expansion are then processed in the same way by recursing the function with the decomposed Boolean function as input. When the logic can be implemented as an unbalanced function, such as in the unmultiplied term of the Davio expansion or as balanced logic gates, that is relayed to the next level of the recursion via an input.

If it is “False”, we have a False-controlled-NOT, meaning an identity function. In this case we remove the term with this function entirely.

Note that with this decomposition, when the function evaluates as “True”, we end up with a True-controlled-NOT, meaning, a NOT function. With the multiplied terms such as the $x_k \cdot F_{x_k=1}$ in Shannon’s Decomposition or $\overline{x_k} \cdot (F_{x_k=0} \oplus F_{x_k=1})$ in the Davio expansion, we simply get x_k or $\overline{x_k}$ respectively. This means we can just implement the side with a True expression using an x_k or $\overline{x_k}$ controlled CNOT gate.

As the recursion terminates, the T-counts of each of the Shannon, Positive Davio, or Negative Davio expansions is compared with each other, and the minimum is chosen. The result is then passed on to the recursion level above it. The process is then repeated with all of the variable orders, and the minimum is picked from the results.

Example 3: We again attempt to implement the function $F(x_3, x_2, x_1) = x_3x_2x_1 + \overline{x_3}(x_1 + x_2)$ as its relative phase

Algorithm 2 lut_synthesis

```

lut_net ← a LUT network
esop (Boolean Function) ← a method to generate an exact QC to calculate
a Boolean function implemented as a single-target gate
node_traversal ← a method to traverse lut_net
while node_traversal returns a valid node do
  if node is intermediate node then
    QC ← add relative_phase(node’s Boolean function) acting on a dirty
    ancilla
    QC ← add inverse of relative_phase(node’s Boolean function) where
    we restore the state of the same dirty ancilla
  else if node is output node then
    QC ← add esop (node’s Boolean function) acting on a dirty ancilla
  end if
end while

```

equivalent. Just as before, we first take the positive Davio expansion about x_3 (for simplicity, we will only follow the Davio branches in this example. However, it will be easy to check later that it is optimal for this function). We get $F_{x_3=0} = x_1 + x_2$, $F_{x_3=1} = x_1 \cdot x_2$, and $F_{2x_3} = F_{x_3=0} \oplus F_{x_3=1} = (x_1 + x_2) \oplus x_1 \cdot x_2 = x_1 \oplus x_2$. Note that for this level, we can tolerate an unbalanced function, so for the $F_{x_3=0}$ branch, we set the “balanced” parameter to false and on the F_{2x_3} branch we set it to true.

We then proceed to x_2 . We decompose $F_{x_3=0} = F_{x_3=0, x_2=0} \oplus x_2 \cdot F_{2x_3=0, x_2}$ where $F_{2x_3=0, x_2} = F_{x_3=0, x_2=0} \oplus F_{x_3=0, x_2=1}$. Since $F_{x_3=0, x_2=0} = x_1$, we just implement it with an x_1 controlled CNOT. After calculating $F_{x_3=0, x_2=1} = x_1 + 1 = 1$, we get $F_{2x_3=0, x_2} = F_{x_3=0, x_2=0} \oplus F_{x_3=0, x_2=1} = x_1 \oplus 1 = \overline{x_1}$. Thus $x_2 \cdot F_{2x_3=0, x_2}$ is implemented according to a balanced function multiplication such as in Fig. 5. We then compose these according to Fig. B5. We can see at this point that there are four T-gates.

We now go to the F_{2x_3} branch. Here, since $x_1 \oplus x_2$ can be implemented using an x_1 controlled CNOT and x_2 controlled CNOT in series, we synthesize that and terminate the recursion.

Back at the top level, we compose the subcircuits for $F_{x_3=0}$ and F_{2x_3} as a negative unbalanced Davio construction such as in Fig. B8, adding an additional four T-gates. This brings us to a total of 8 T-gates. Compare this to the 24 T-gates of using only Shannon’s Decomposition, 1/3 the size. We omit the computation of the other branches and variable orders in the interest of space, but the reader can perform them easily to see that the branch we followed is the optimal branch (or one of them). In this function, there are multiple because x_1 and x_2 are symmetric, and several Davio branches have similar costs). The resulting circuit is depicted in Fig. A2 in Appendix A.

3.6 Synthesizing A Boolean Function Using Boolean logic networks and Shannon’s decomposition

Using Algorithm 1 we are now ready to propose our LUT-based synthesis method, described in Algorithm 2.

Like Sec. 2.5, we query a node traversal algorithm to get the next node. We then check if this node is an output node or an intermediate node. If it is an intermediate node,

we generate the compute logic as a single-target gate using Algorithm 1, and then invert that output to generate the uncompute logic, then add those gates to the circuit. If the node is an output node, we generate the single-target gate using exact logic and add it to the circuit. We continue until there are no more nodes to process from the node traversal algorithm and terminate.

4. Experimental Results

4.1 Implementation

Boolean functions are taken from the EPFL Combinational Benchmark Suite [8] and Boolean logic networks are generated from them using the ABC tool [9]. In particular, we choose circuits with numbers of primary inputs much larger than 6 to get a decent number of intermediate nodes. We synthesize these Boolean logic networks with LUT-sizes of 3, 4, 5, and 6 variables for each node, which we will denote LUT3, LUT4, LUT5, and LUT6 respectively. As in [2], this represents a range where the T-count remains tractable. This Boolean logic network is converted to a quantum circuit implemented in Qiskit [10], including any uncomputation logic.

The algorithm is implemented in Python. Any exact (non-relative phase) synthesis of Boolean functions is conducted using Qiskit’s ClassicalFunction functionality, which uses the ESOP-based PKRM algorithm [7].

In our implementation, we do an exhaustive search by executing our algorithm through every variable order to find the one with the lowest T-count. As we can see from Table 2, this is tractable for $n < 6$.

4.2 Methodology

The results generated using the *Proposed method* are then compared to a naive implementation of the LUT-based synthesis method (*ESOP-based method*), where each LUT is synthesized as an exact (non relative phase) implementation of their Boolean functions. We also run a version of the algorithm described in Sec. 3.2 using only Shannon’s decomposition (Shannon Only), to test the efficacy of merely using a naive relative phase implementation without the optimization of introducing the Davio’s expansions and balanced/unbalanced constructions. In the ESOP-based method, because the relative phase functions are constructed without any ancilla, we assume that there are plenty of bits to be reused at any time. These contain MPMCT (Multiple Polarity Multiple Control Toffoli) gates, which are MCT gates that also implement functions including negated inputs e.g. $F(x_0, x_1, x_2) = x_0 \bar{x}_1 x_2$ etc. This means that any such MPMCT gates are either synthesized directly using a Gray-code sequence of operations for $n < 5$, or synthesized according to the Barenco dirty ancilla method otherwise (both are described in [2]). This same decomposition method is also used in the proposed method for any output node that requires an exact implementation.

As in [2], we note that for values of $n < 5$, the T-count in a direct implementation of the Toffoli gate (i.e. using a Gray-code sequence of operations) scales with 2^{n-1} .

Note that because, according to [6], the number of ancilla is completely determined by the logic network and the traversal algorithm we use there, the ancilla will be identical across ESOP-based, Shannon-Only and Proposed. As we are only concerned with the relative performance of the three methods, we do not include them in the table. However, because changing the LUT size also changes the logic network, there will be some corresponding change in the number of ancilla used. We note the effect of this on ancilla in qualitative terms later.

Our experimental results are displayed in Table 1. We compare the ratio of the T-count for each mapped LUT generated by either the ESOP-based method or the proposed method with that of LUT6, which is the upper bound of the range of variable lengths of tractable T-count proposed in [2], in the second column generated by the ESOP-based method as a baseline. The LUT6 should give us the largest T-count for each node, and having the largest nodes should give us the least amount of ancilla. Recall that in the previous paragraph, we noted that the T-count of each relative phase LUT scales with 2^{n-1} . Synthesizing using Boolean logic networks with different LUT-sizes measures the contribution of decreasing the LUT-size at the cost of increasing the number of gates and ancilla used. In particular, this is useful for comparison because it measures how the tradeoff between T-count and ancilla relates to the Shannon-Only and Proposed methods. For each columns LUT3, LUT4, LUT5, and LUT6 under each method, we take the arithmetic mean of the values of each column right below it in the “Per LUT Average” row. For each method, we take the arithmetic average of the columns of the Per LUT Average row it in the “Average” row for that method. Note, however, that these numbers exempt `adder_size_2022` from their numbers: we will explain why this circuit is interesting and also why it should not be included in these averages in Sec. 4.3. Additionally, we display the average runtimes for each circuit for each iteration of the Proposed method in Table 2.

4.3 Analysis

In Table 1, the best result for each circuit is written in bold. Just as before, the Proposed method sees the largest gain over the baseline LUT6 ESOP-based numbers in LUT3. However, whereas the Shannon Only method saw only modest gains here over the ESOP-based LUT3 numbers (i.e., merely reducing the maximum LUT-size with the naive method), we see a much more significant advantage for the Proposed method. Of note, Proposed method shows an absolute advantage over the Shannon Only method over all the cases tested.

Of particular note is `bar_size_2015`, which contains many highly complex, maximum size nodes. The Shannon Only method suffered from its 2^{n-1} proportionality here and severely underperformed the ESOP-based method. How-

Table 1: Experimental Results

Circuit	ESOP-based method				Shannon-Only				Proposed method			
	LUT6 (T-count)	LUT5	LUT4	LUT3	LUT6	LUT5	LUT4	LUT3	LUT6	LUT5	LUT4	LUT3
priority_size_2022	98664	0.56	0.32	0.26	0.27	0.51	0.19	0.16	0.14	0.08	0.06	0.05
int2float_size_2022	22592	1.14	0.49	0.38	0.94	0.94	0.4	0.35	0.27	0.17	0.1	0.06
i2c_size_2022	89288	0.69	0.4	0.27	0.94	0.68	0.31	0.23	0.61	0.27	0.18	0.13
dec_size_2018	57344	0.5	0.41	0.2	1	0.41	0.31	0.16	1	0.39	0.3	0.15
ctrl_size_2022	7560	1.17	0.84	0.68	0.99	1.12	0.51	0.41	0.99	0.88	0.35	0.24
cavlc_size_2022	116896	0.8	0.49	0.41	0.63	0.6	0.3	0.29	0.13	0.1	0.07	0.06
bar_size_2015	176400	0.94	0.03	0.03	7.15	1.4	0.21	0.21	0.15	0.21	0.07	0.07
arbiter_size_2022	175640	0.68	0.42	0.28	0.35	0.63	0.27	0.17	0.21	0.13	0.11	0.05
adder_size_2022	4992	9.75	6.38	1.94	1	7.16	4.06	2.54	1	1.91	1.95	1.66
Per LUT Average		1.80	1.08	0.49	0.5	0.46	0.35	0.27	1.47	1.49	0.73	0.50
Average			1.13			0.72				0.24		

Table 2: Runtime of Proposed Method

Circuit	CPU time of the proposed method (sec)			
	LUT6	LUT5	LUT4	LUT3
priority_size_2022	14.01	4.61	0.33	1.03
int2float_size_2022	10.57	2.03	0.94	0.6
i2c_size_2022	89.33	18.57	2.17	0.71
dec_size_2018	5.14	0.33	1.17	1.21
ctrl_size_2022	9.87	4.52	0.45	0.37
cavlc_size_2022	16.46	3.24	2.72	0.22
bar_size_2015	340.25	119.99	1.46	0.99
arbiter_size_2022	33.74	1.78	4.01	0.02
adder_size_2022	129.88	63.09	2.37	1.07

ever, the Proposed method shows a better ability to break down the complex functions into something with tractable T-count. This seems to be due to the introduction of the Davio expansion and its ability to create a more natural ESOP expansion of the original functions, which lends itself to fewer multiplications, and thus to fewer T-gates used.

However, adder_size_2022 remains an exception because it is a particularly interesting case. At LUT6 size, all of the logic for every output bit fits into a single node, and thus, as is evident from the table, both the Proposed method and Shannon Only method at LUT6 are identical to the naive case. What is more telling is that reducing the LUT-size does NOT help in this case. While the Proposed method outperforms both the ESOP-based method and Shannon Only method at smaller LUT-sizes, it still remains inferior to the LUT6 case. Speaking in terms of the Boolean logic network, the LUT6 case is simply a set of independent output nodes. An interesting further topic of research would be to synthesize Boolean logic networks that explicitly aim for this type of construction and again do the same experiments.

5. Conclusion and Future Research

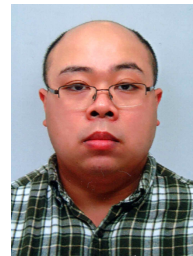
This paper proposed the use of Shannon's Decomposition, Davio expansions and alternating balanced/unbalanced relative phase constructions to optimize relative phase constructions in their usage in LUT-based quantum circuit synthesis. The experimental results showed a clear advantage over the naive case, and Davio Expansions and balanced/unbalanced constructions prove effective at dealing with corner cases that hinder the Shannon Only case. There remains one interesting exception where the proposed method underperforms the naive method, however, and it relates to a special form of Boolean logic network which consists of independent output gates.

The results of this paper show a clear advantage to using our proposed method, as well as possible further avenues of

inquiry for related research. As the experimental results show, there remain many variables to control to optimize in this method, and researching those further could prove fruitful.

References

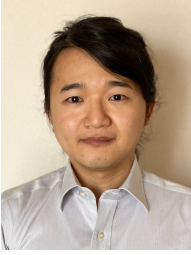
- [1] S. Yamashita, S. Minato, and D.M. Miller, "DDMF: An Efficient Decision Diagram Structure for Design Verification of Quantum Circuits Under a Practical Restriction," IEICE transactions on fundamentals of electronics, communications and computer sciences, vol.91, no.12, pp.3793–3802, 2008.
- [2] A. Barenco, C.H. Bennett, R. Cleve, D.P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J.A. Smolin, and H. Weinfurter, "Elementary Gates for Quantum Computation," Phys. Rev. A, vol.52, pp.3457–3467, Nov 1995.
- [3] D. Maslov, "Advantages of Using Relative-phase Toffoli Gates with an Application to Multiple Control toffoli Optimization," Physical Review A, vol.93, no.2, p.022311, 2016.
- [4] B. Giles and P. Selinger, "Exact Synthesis of Multiqubit Clifford+*t* Circuits," Phys. Rev. A, vol.87, p.032332, Mar 2013.
- [5] M. Amy and N.J. Ross, "Phase-state Duality in Reversible Circuit Design," Phys. Rev. A, vol.104, p.052602, Nov 2021.
- [6] M. Soeken, M. Roetteler, N. Wiebe, and G. De Micheli, "LUT-based Hierarchical Reversible Logic Synthesis," IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol.38, no.9, pp.1675–1688, 2018.
- [7] R. Drechsler, "Pseudo-Kronecker Expressions for Symmetric Functions," IEEE Transactions on Computers, vol.48, no.9, pp.987–990, 1999.
- [8] L. Amaru, P.E. Gaillardon, and G. De Micheli Proceedings of the 24th International Workshop on Logic & Synthesis (IWLS), 2015.
- [9] R. Brayton and A. Mishchenko, "ABC: An Academic Industrial-strength Verification Tool," Computer Aided Verification, ed. T. Touili, B. Cook, and P. Jackson, Berlin, Heidelberg, pp.24–40, Springer Berlin Heidelberg, 2010.
- [10] Qiskit contributors, "Qiskit: An Open-source Framework for Quantum Computing," 2023.



David Clarino received his BS EECS and BA Physics from UC Berkeley and MS from Santa Clara University. He is currently a student at the Graduate School of Information Science and Engineering, Ritsumeikan University, Shiga, Japan. His research interests include quantum circuit design and quantum circuit verification, and he has previously worked on SoC design verification.



Naoya Asada received his bachelor's degree from College of Information Science and Engineering, Ritsumeikan University in 2021. He received his master's degree from Graduate School of Information Science and Engineering, Ritsumeikan University in 2023. He is currently with Mitsubishi Electric Corp.



Atsushi Matsuo received his B.E., M.E. and Ph.D. degrees in Information Science and Engineering from Ritsumeikan University, Shiga, Japan in 2011, 2013, and 2023, respectively. His research interests include quantum circuit design, compilers for quantum circuits, and variational quantum algorithms.



Shigeru Yamashita is a professor at the Department of Computer Science, College of Information Science and Engineering, Ritsumeikan University. He received his B.E., M.E. and Ph.D. degrees in Information Science from Kyoto University, Kyoto, Japan, in 1993, 1995 and 2001, respectively. His research interests include new types of computation and logic synthesis for them. He received the 2000 IEEE Circuits and Systems Society Transactions on Computer-Aided Design of Integrated Circuits and Systems

Best Paper Award, SASIMI 2010 Best Paper Award, 2010 IPSJ Yamashita SIG Research Award, and 2010 Marubun Academic Achievement Award of the Marubun Research Promotion Foundation. He is a senior member of IEEE and a senior member of IPSJ.

Appendix A: Function Decomposition

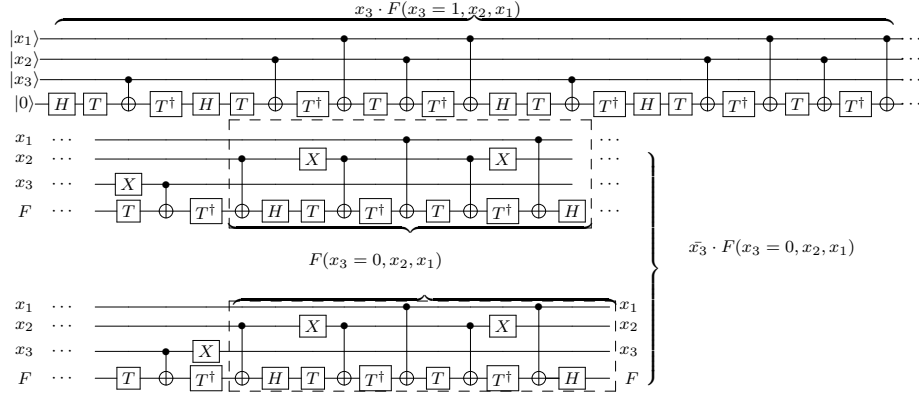


Fig. A1: Decomposition of the function in Example 2

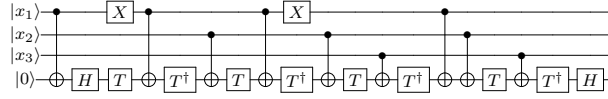


Fig. A2: Decomposition of the function in Example 3

Appendix B: Various Balanced and Unbalanced Shannon's and Davio Decompositions from Section 3.5

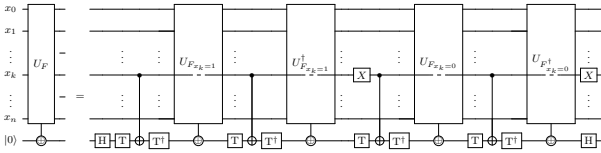


Fig. B3: Balanced Shannon's Decomposition

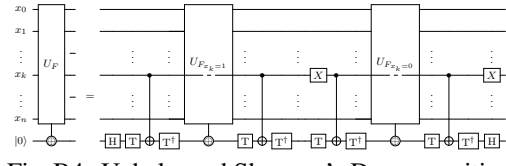


Fig. B4: Unbalanced Shannon's Decomposition

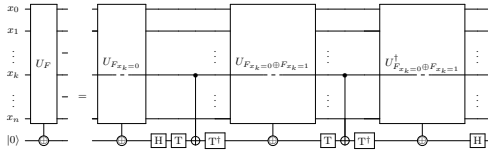


Fig. B5: Balanced positive Davio Expansion

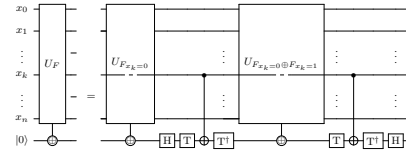


Fig. B6: Unbalanced positive Davio Expansion

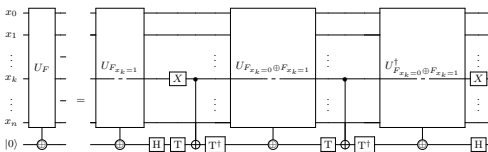


Fig. B7: Balanced negative Davio Expansion

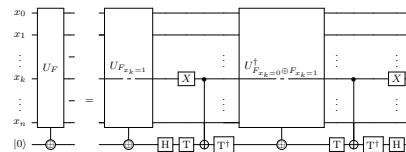


Fig. B8: Unbalanced negative Davio Expansion